

@supplier_scout

Appears in: [DL-02](#)

orchestra/agents/supplier_scout.py:17, 20-138

(same parallel-agents sentence as above)

@supplier_scout

orchestra/agents/supplier_scout.py:17-17

17 PRICE_SPIKE_THRESHOLD = 0.30

...

orchestra/agents/supplier_scout.py:20-138

```
20  def run(brief: dict, llm=None) -> list:
21      """Returns list of Finding dicts."""
22      if llm is None:
23          llm = get_llm()
24
25      inventory = brief["inventory"]
26      suppliers = brief["suppliers"]
27      all_pos = brief.get("all_pos", [])
28
29      matrix = _build_matrix(inventory, suppliers)
30      findings = []
31
32      # Price spike detection
33      sku_prices: dict = {}
34      for po in all_pos:
35          sku_prices.setdefault(po["sku_id"], []).append(po["unit_price"])
36
37      for sku_id, prices in sku_prices.items():
38          avg = sum(prices) / len(prices)
39          options = matrix.get(sku_id, [])
40          quotes = [o["unit_price"] for o in options if o["available"]]
41          if not quotes:
42              continue
43          best = min(quotes)
44          if best > avg * (1 + PRICE_SPIKE_THRESHOLD):
45              pct = (best / avg - 1) * 100
46              findings.append({
47                  "agent": AGENT_NAME,
48                  "type": "price_spike",
49                  "severity": "warning",
50                  "sku_id": sku_id,
51                  "confidence": 0.0,
52                  "data": {
53                      "best_quote": round(best, 2),
54                      "historical_avg": round(avg, 2),
55                      "pct_above": round(pct, 1),
56                      "options": options,
57                  },
58                  "reasoning": (
59                      f"Best quote ${best:.2f} is {pct:.0f}% above "
60                      f"historical avg ${avg:.2f}. Human approval required."
61                  ),
62                  "recommendation": "flag_for_approval",
63              })
64
65      # Unavailability detection for stockout SKUs
66      critical_skus = {
67          item["sku_id"]
68          for item in inventory
69          if item["current_stock"] < item["reorder_point"]
70      }
71      for sku_id in critical_skus:
72          options = matrix.get(sku_id, [])
73          if options and not any(o["available"] for o in options):
74              findings.append({
75                  "agent": AGENT_NAME,
76                  "type": "supplier_unavailable",
77                  "severity": "critical",
78                  "sku_id": sku_id,
79                  "confidence": 1.0,
80                  "data": {"options": options},
81                  "reasoning": f"All {len(options)} supplier(s) unavailable for {sku_id}.",
82                  "recommendation": "escalate",
83              })
84
85      # LLM ranking for critical SKUs that have available suppliers
86      for sku_id in critical_skus:
87          options = [o for o in matrix.get(sku_id, []) if o["available"]]
88          if not options:
89              continue
90          item_data = next((i for i in inventory if i["sku_id"] == sku_id), {})
91          daily = max(item_data.get("daily_usage_avg", 1), 0.1)
92          days = round(item_data.get("current_stock", 0) / daily, 1)
93
94          result = call_json(
95              llm,
96              system=(
97                  "You are a supplier evaluation specialist. Return ONLY valid JSON -- no markdown:\n"
98                  '{"recommended_supplier_id": "...", "confidence": 0.0-1.0, '
99                  '"tradeoff_summary": "...", "red_flags": []}'
100          ),
101          user=(
102              f"SKU needed: {sku_id}\n"
103              f"Urgency: {days} days of stock remaining\n\n"
104              f"Available suppliers (sorted by score = reliability / lead_days):\n"
105              f'{json.dumps(options, indent=2)}\n\n'
106              "Select the best supplier. Explain the reliability vs speed vs cost tradeoff. "
107              "Flag any concerns."
108          ),
109          )
110
111          if result.get("fallback"):
112              rec = options[0]
113              tradeoff = f"Rule-based: {rec['name']} (score={rec['score']})"
114              red_flags = []
115              confidence = 0.7
116          else:
117              rec_id = result.get("recommended_supplier_id", options[0]["supplier_id"])
118              rec = next((o for o in options if o["supplier_id"] == rec_id), options[0])
119              tradeoff = result.get("tradeoff_summary", "")
120              red_flags = result.get("red_flags", [])
121              confidence = float(result.get("confidence", 0.8))
122
123          findings.append({
124              "agent": AGENT_NAME,
125              "type": "supplier_recommendation",
126              "severity": "info",
127              "sku_id": sku_id,
128              "confidence": confidence,
129              "data": {
130                  "recommended_supplier": rec,
131                  "all_options": options,
132                  "red_flags": red_flags,
133              },
134              "reasoning": tradeoff,
135              "recommendation": "use_supplier",
136          })
137
138      return findings
```

What it shows: Specialist agent #2: scores suppliers (reliability / lead_days), detects price spikes (>30%) and unavailability, and LLM-ranks options for critical SKUs.

Source: [orchestra/agents/supplier_scout.py:17, 20-138](#) · image rendered by [build_snippets.py](#)